

Reviewer Pack — Minimal p-adic Cohomological Dimension and SAT Clause Entrop...

Entry 1d1ea847605e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 03:24:47 UTC

Minimal p-adic Cohomological Dimension and SAT Clause Entropy Correlation

Entry ID: 1d1ea847605e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 03:24:47 UTC

1. Conjecture

Field A (mathematical branch): p-adic Hodge Theory **Field B** (complexity object): SAT Clause Sets

Statement:

For any given CNF φ with n variables, the minimal p-adic cohomological dimension ($\text{mcd}(\varphi)$) of its associated variety is linearly correlated with its clause entropy ($h(\varphi)$), such that $\text{mcd}(\varphi) = \Theta(h(\varphi))$.

Rationale (proposer's reasoning):

The relationship between algebraic properties of varieties and combinatorial properties of SAT instances suggests a potential bridge in understanding the complexity of proof search. p-adic cohomology provides a rich algebraic invariant, while clause entropy measures the randomness in the structure of clauses, which may correlate with the difficulty of finding a satisfying assignment.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4b0a473c0bdc10d5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\text{mcd}(\varphi)$ and $h(\varphi)$ across 30 random seeds is ≥ 0.8 , and the mean absolute difference between $\text{mcd}(\varphi)$ and $h(\varphi)$ for all φ is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, h):
        cnf = []
        for _ in range(h):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def compute_clause_entropy(cnf):
        total_clauses = len(cnf)
        entropy = 0
        for i in range(total_clauses):
            for j in range(i + 1, total_clauses):
                if set(cnf[i]) & set(cnf[j]):
                    continue
                p = Fraction(1, total_clauses - i)
                entropy += math.log2(p) * p
        return entropy

    def compute_mcd(cnf):
        n = len(set(abs(lit) for clause in cnf for lit in clause))
        return n

    n_values = [5, 10, 15, 20, 30, 40]
    instances_tested = 0
    total_mcd = 0
    total_entropy = 0

    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            cnf = generate_cnf(n, random.randint(1, n))
            entropy = compute_clause_entropy(cnf)
            mcd = compute_mcd(cnf)
```

```

        total_mcd += mcd
        total_entropy += entropy
        instances_tested += 1

mean_mcd = total_mcd / instances_tested
mean_entropy = total_entropy / instances_tested
correlation_coefficient = (instances_tested * sum(mcd * entropy for mcd, entropy in zip([mean_mcd,
sum(mcd for mcd in [mean_mcd] * instances_tested) * sum(entropy for
math.sqrt((instances_tested * sum(mcd**2 for mcd in [mean_mcd] * ins
sum(mcd for mcd in [mean_mcd] * instances_tested)**2) *
(instances_tested * sum(entropy**2 for entropy in [mean_en
sum(entropy for entropy in [mean_entropy] * instances_tested

mean_abs_diff = abs(mean_mcd - mean_entropy)

return {
    "metric_name": "mcd_vs_entropy",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.8 and mean_abs_diff <= 3,
    "counterexample": "" if correlation_coefficient >= 0.8 and mean_abs_diff <= 3 else f"mcd={
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 59)) # Default to first 30 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support_
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mcd_vs_entropy\" first_failing_seed={first_fail
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {"seed": 11, "metric_name": "mcd_vs_entropy", "metric_value": 0.7071, "instances_tested": 30,
20.661172227788906"}
```

```
--STDERR--
```

[illegible]

8. Critic adversarial review

Critic reasoning:

9. Final verdict & safety rail

Reasoning:

11. Audit log (LLM calls)

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21888
2	preregistration	ollama_remote	glm4:latest	0	0	9400
3	novelty	ollama_remote	glm4:latest	0	0	8536
4	novelty	ollama_remote	glm4:latest	0	0	8996
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14541
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12897
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10945
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19057
9	judge	ollama_remote	glm4:latest	0	0	13854

(full prompt+response transcripts available in [research/audit/1d1ea847605e.jsonl](#))

12. Reproducibility

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1d1ea847605e.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1d1ea847605e.tar.gz (if generated)